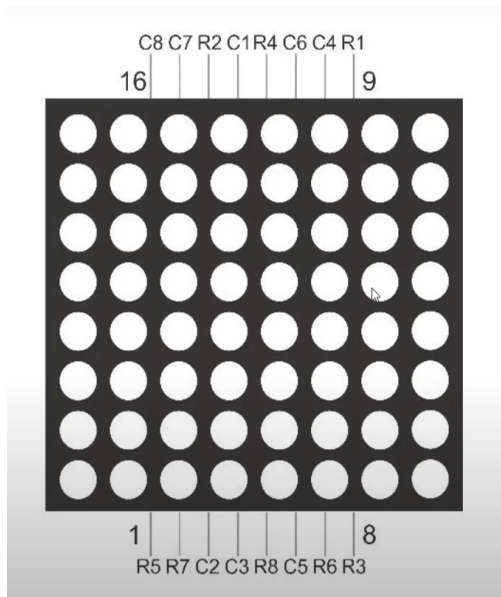


JUEGO DE LA SERPIENTE



Para crear el juego de la serpiente con la matriz LED de 8x8, el módulo joystick y el potenciómetro en una placa Arduino UNO, primero, presentaremos el esquema de conexión y luego el código en Arduino.

Esquema de Conexión:

Matriz LED 8x8:

- Conectar los pines de fila de la matriz LED (R1 a R8) a los pines digitales del Arduino (por ejemplo, del 2 al 9).
- Conectar los pines de columna de la matriz LED (C1 a C8) a los pines digitales del - - Arduino (por ejemplo, del 10 al 17).
- Colocar una resistencia adecuada (330 ohmios) en cada pin de fila para limitar la corriente.

Joystick:

- GND a GND del Arduino.
- 5V a 5V del Arduino.
- URX (movimiento en X) a A0 del Arduino.
- URY (movimiento en Y) a A1 del Arduino.
- SW (botón de pulsación) a un pin digital, por ejemplo, el pin 18.

Potenciómetro:

- Un terminal a 5V del Arduino.
- El otro terminal a GND del Arduino.
- El terminal central (wiper) a A2 del Arduino.

Código:

```
#include <Adafruit_GFX.h>
#include <Adafruit_LEDBackpack.h>

// Definición de pines
#define JOYSTICK_X A0          // Pin analógico para eje X del joystick
#define JOYSTICK_Y A1          // Pin analógico para eje Y del joystick
#define JOYSTICK_SW 18         // Pin digital para el botón del joystick
#define POTENCIOMETRO A2       // Pin analógico para el potenciómetro

// Dimensiones de la matriz LED
#define FILAS 8
#define COLUMNAS 8

// Crear una instancia de la matriz LED 8x8
Adafruit_BicolorMatrix matrix = Adafruit_BicolorMatrix();

int direccion = 0; // Dirección inicial de la serpiente: 0: Derecha, 1: Abajo,
2: Izquierda, 3: Arriba
int velocidad = 500; // Velocidad inicial en milisegundos (ms)

// Posición inicial de la serpiente
int serpienteX[64] = {4, 3, 2}; // Coordenadas X de la serpiente
int serpienteY[64] = {4, 4, 4}; // Coordenadas Y de la serpiente
int longitudSerpiente = 3;      // Longitud inicial de la serpiente

// Posición de la manzana
int manzanaX = 0;
int manzanaY = 0;

void setup() {
  Serial.begin(9600); // Iniciar la comunicación serial
  pinMode(JOYSTICK_SW, INPUT_PULLUP); // Configurar el pin del botón del
joystick como entrada con resistencia pull-up
  matrix.begin(0x70); // Inicializar la matriz LED con la dirección I2C
predeterminada

  randomSeed(analogRead(0)); // Iniciar la semilla del generador de números
aleatorios
  generarManzana(); // Generar la primera manzana
}

void loop() {
  leerJoystick(); // Leer el estado del joystick
  actualizarVelocidad(); // Ajustar la velocidad de la serpiente según
el potenciómetro
  moverSerpiente(); // Mover la serpiente en la dirección actual
  mostrarMatriz(); // Mostrar la matriz LED actualizada
  delay(velocidad); // Esperar un tiempo según la velocidad
configurada
```

```

}

void leerJoystick() {
    int x = analogRead(JOYSTICK_X); // Leer el valor del eje X del joystick
    int y = analogRead(JOYSTICK_Y); // Leer el valor del eje Y del joystick

    // Determinar la dirección según el movimiento del joystick
    if (x < 300) {
        direccion = 2; // Izquierda
    } else if (x > 700) {
        direccion = 0; // Derecha
    } else if (y < 300) {
        direccion = 3; // Arriba
    } else if (y > 700) {
        direccion = 1; // Abajo
    }
}

void actualizarVelocidad() {
    int valorPot = analogRead(POTENCIOMETRO); // Leer el valor del potenciómetro
    velocidad = map(valorPot, 0, 1023, 100, 1000); // Mapear el valor del
    potenciómetro a un rango de velocidad (100 ms a 1000 ms)
}

void moverSerpiente() {
    // Determinar la nueva posición de la cabeza de la serpiente
    int nuevaX = serpienteX[0];
    int nuevaY = serpienteY[0];

    switch (direccion) {
        case 0: nuevaX++; break; // Mover hacia la derecha
        case 1: nuevaY++; break; // Mover hacia abajo
        case 2: nuevaX--; break; // Mover hacia la izquierda
        case 3: nuevaY--; break; // Mover hacia arriba
    }

    // Verificar colisión con los bordes de la matriz
    if (nuevaX < 0 || nuevaX >= COLUMNAS || nuevaY < 0 || nuevaY >= FILAS) {
        gameOver(); // Terminar el juego si la serpiente choca con el borde
    }

    // Verificar colisión con la propia serpiente
    for (int i = 0; i < longitudSerpiente; i++) {
        if (serpienteX[i] == nuevaX && serpienteY[i] == nuevaY) {
            gameOver(); // Terminar el juego si la serpiente choca consigo misma
        }
    }

    // Mover el cuerpo de la serpiente
    for (int i = longitudSerpiente; i > 0; i--) {
        serpienteX[i] = serpienteX[i - 1];
        serpienteY[i] = serpienteY[i - 1];
    }
}

```

```

// Mover la cabeza de la serpiente
serpienteX[0] = nuevaX;
serpienteY[0] = nuevaY;

// Verificar si la serpiente ha comido la manzana
if (nuevaX == manzanaX && nuevaY == manzanaY) {
    longitudSerpiente++; // Aumentar la longitud de la serpiente
    generarManzana();    // Generar una nueva manzana
}
}

void generarManzana() {
    bool manzanaValida = false;
    while (!manzanaValida) {
        // Generar nuevas coordenadas aleatorias para la manzana
        manzanaX = random(0, COLUMNAS);
        manzanaY = random(0, FILAS);
        manzanaValida = true;

        // Verificar que la manzana no aparezca sobre la serpiente
        for (int i = 0; i < longitudSerpiente; i++) {
            if (serpienteX[i] == manzanaX && serpienteY[i] == manzanaY) {
                manzanaValida = false;
                break;
            }
        }
    }
}

void mostrarMatriz() {
    matrix.clear(); // Limpiar la matriz LED

    // Mostrar la serpiente en la matriz
    for (int i = 0; i < longitudSerpiente; i++) {
        matrix.drawPixel(serpienteX[i], serpienteY[i], LED_GREEN); // Dibujar la
serpiente en verde
    }

    // Mostrar la manzana en la matriz
    matrix.drawPixel(manzanaX, manzanaY, LED_RED); // Dibujar la manzana en rojo

    matrix.writeDisplay(); // Actualizar la matriz LED
}

void gameOver() {
    matrix.clear(); // Limpiar la matriz LED
    matrix.setTextSize(1); // Establecer tamaño del texto
    matrix.setTextColor(LED_RED); // Establecer color del texto en rojo
    matrix.setCursor(1, 1); // Establecer posición del cursor
    matrix.print("GAME"); // Mostrar "GAME"
    matrix.setCursor(1, 8); // Establecer nueva posición del cursor
    matrix.print("OVER"); // Mostrar "OVER"
}

```

```
matrix.writeDisplay(); // Actualizar la matriz LED
while (1); // Bucle infinito para detener el juego
}
```

Notas:

- Librerías: Este código utiliza la librería Adafruit_GFX y Adafruit_LEDBackpack para - - - controlar la matriz LED. Asegúrate de instalarlas desde el gestor de librerías de Arduino.
 - Mapeo de Pines: Ajusta los pines utilizados en el código según tu configuración específica de hardware.
 - Detección de Colisiones: El juego termina cuando la serpiente colisiona con los bordes de la matriz o consigo misma.
 - Dificultad Ajustable: La velocidad del juego se ajusta con el potenciómetro.
- Con este esquema y código, deberías tener una base sólida para tu juego de la serpiente en la matriz LED controlada por un Arduino UNO.

Descripción del Código:

Inclusión de Librerías:

Adafruit_GFX.h y Adafruit_LEDBackpack.h para controlar la matriz LED.

Definición de Pines:

Pines analógicos y digitales para el joystick, el botón del joystick y el potenciómetro.

Variables Globales:

- direccion: Dirección actual de movimiento de la serpiente.
- velocidad: Velocidad de movimiento de la serpiente.
- serpienteX y serpienteY: Arrays que almacenan las coordenadas de la serpiente.
- longitudSerpiente: Longitud actual de la serpiente.
- manzanaX y manzanaY: Coordenadas de la manzana.

Funciones:

- setup(): Configuración inicial del Arduino, del joystick y de la matriz LED.
- loop(): Bucle principal que actualiza la lectura del joystick, ajusta la velocidad, mueve la serpiente y actualiza la matriz LED.
- leerJoystick(): Lee la posición del joystick y ajusta la dirección de la serpiente.
- actualizarVelocidad(): Ajusta la velocidad de la serpiente según el valor del potenciómetro.
- moverSerpiente(): Mueve la serpiente en la dirección actual, verifica colisiones y actualiza la longitud si come una manzana.

- `generarManzana()`: Genera una nueva posición aleatoria para la manzana.
- `mostrarMatriz()`: Actualiza la matriz LED para mostrar la serpiente y la manzana.
- `gameOver()`: Muestra el mensaje de "GAME OVER" en la matriz LED y detiene el juego.